PYTHON · CHULETA BÁSICA

Tipos, colecciones, métodos y slicing

DATOS SIMPLES Y CONVERSIONES

```
n = 12             # int: entero
precio = 19.95     # float: decimal
marca = "Seat"     # str: texto
activo = True      # bool: True / False
dato = None        # ausencia de valor
int("12")          # 12
float("3.5")       # 3.5
str(12)            # "12"
bool(0)            # False
type(precio)       # float
isinstance(n, int) # True
```

False, None, 0 y las colecciones vacías son falsos en una condición. El resto, en general, verdadero.

ENTRADA, SALIDA Y F-STRINGS

```
texto = input("Modelo: ")  # siempre str
n = int(input("Cantidad: "))
print("Modelo:", texto)
precio = 19.95
print(f"Total: {precio:.2f} €")
```

f"...{expresión}..." inserta valores. :.2f muestra dos decimales. Una conversión incorrecta puede dar ValueError.

OPERADORES

Aritméticos: + suma; - resta; * multiplica.
/ divide; // redondea hacia abajo; % resto; ** potencia.
7 / 2 → 3.5; 7 // 2 → 3; 7 % 2 → 1.
Asignación: = asigna; += suma; -= resta.
*= multiplica; /= divide; //= divide hacia abajo.
%= resto; **= potencia (con números).
x += 2 equivale a x = x + 2.
Comparación: devuelve True o False
== igual; != distinto; < menor; > mayor.
<= menor o igual; >= mayor o igual.
Lógica: and (ambas), or (alguna), not (niega).
Pertenencia: in, not in. **Identidad:** is, is not.
dato is None comprueba la ausencia de valor.

STR: MÉTODOS DE TEXTO

```
s = "  Seat Ibiza  "
s.strip()          # quita espacios extremos
s.lower(); s.upper()
s.replace("Seat", "Ford")
s.split()          # ["Seat", "Ibiza"]
"-".join(["a", "b"])  # "a-b"
s.find("Ibiza")    # posición; -1 si no está
s.count("a")       # cuenta apariciones
s.startswith("  S")
s.endswith("  ")
"123".isdigit()    # True
```

El texto es inmutable: estos métodos devuelven resultados. Para conservar un cambio: s = s.strip().

ÍNDICES Y SLICING

En str, list y tuple: **secuencia[inicio:fin:paso]**. El fin queda fuera.

```
a = [10, 20, 30, 40, 50]
a[0]       # 10: primero
a[-1]      # 50: último
a[1:4]     # [20, 30, 40]
a[:3]      # [10, 20, 30]
a[2:]      # [30, 40, 50]
a[::2]     # [10, 30, 50]
a[::-1]    # orden inverso
```

Los índices empiezan en 0. Paso negativo: retrocede. El paso no puede ser 0.

LIST: ORDENADA Y MODIFICABLE

Admite repetidos. Acceso por índice.

```
a = [30, 10, 20]
a[0] = 40          # sustituir
a.append(50)       # añade un elemento
a.extend([60, 70]) # añade varios
a.insert(1, 15)    # inserta en posición 1
a.remove(20)       # borra primera coincidencia
x = a.pop()        # extrae y devuelve último
x = a.pop(0)       # extrae por índice
a.index(50); a.count(50)
a.sort()           # ordena la propia lista
a.reverse()        # invierte la propia lista
b = a.copy(); a.clear()
```

index: primera posición; count: apariciones; copy: copia superficial; clear: vaciar. remove e index fallan si falta el valor. sort y reverse modifican y devuelven None.

TUPLE: ORDENADA E INMUTABLE

```
t = (10, 20, 10)
uno = (10,)        # la coma crea la tupla
t.count(10)        # 2
t.index(20)        # 1
x, y = (10, 20)    # desempaquetar
```

Admite índices y slicing. No permite sustituir, añadir ni eliminar elementos.

DICT: PARES CLAVE → VALOR

Claves únicas. Conserva el orden de inserción.

```
d = {"marca": "Seat", "anio": 2020}
d["marca"]         # acceso; falla si no existe
d.get("km", 0)     # 0 si falta la clave
d["km"] = 12000    # añade o actualiza
d.update({"anio": 2021})
d.keys(); d.values()  # claves; valores
d.items()          # pares (clave, valor)
d.pop("km", None) # extrae; None si falta
del d["anio"]      # borra; falla si no existe
d.copy(); d.clear()   # copiar; vaciar
```

"marca" in d comprueba las claves. Recorrer d directamente también recorre sus claves.

SET: VALORES ÚNICOS, SIN ÍNDICES

```
s = {1, 2, 3}
vacio = set()      # {} crea un diccionario
s.add(4); s.update([4, 5])  # uno; varios
s.remove(2)       # falla si no está
s.discard(9)      # no falla si no está
s.pop()           # extrae uno arbitrario
a, b = {1, 2}, {2, 3}
a | b             # unión: {1, 2, 3}
a & b             # intersección: {2}
a - b             # diferencia: {1}
a ^ b             # valores no compartidos
a <= b            # ¿a es subconjunto de b?
s.copy(); s.clear()
```

No garantiza orden de recorrido. No admite índices ni slicing.

FUNCIONES ÚTILES Y COPIAS

```
a = [30, 10, 20]
len(a); sum(a)   # cantidad: 3; suma: 60
min(a); max(a)   # 10; 30
sorted(a)        # nueva lista ascendente
sorted(a, reverse=True)  # descendente
list("abc"); tuple(a); set(a)
```

b = a: misma lista. **b = a.copy()**: otra lista; si está anidada, comparte los objetos interiores.

PYTHON · CHULETA BÁSICA

Control de flujo, iteración, funciones y objetos

IF / ELIF / ELSE

```
velocidad = 50
if velocidad > 120:
    print("Exceso")
elif velocidad > 0:
    print("En marcha")
else:
    print("Parado")
```

Se ejecuta la primera rama verdadera. Indenta el cuerpo de cada bloque con 4 espacios.

EXPRESIÓN TERNARIA

valor_si_verdadero if condición else valor_si_falso

```
estado = "marcha" if velocidad > 0 else "paro"
```

MATCH / CASE (PYTHON 3.10+)

```
opcion = "ver"
match opcion:
    case "ver" | "listar":
        print("Mostrar coches")
    case "salir":
        print("Fin")
    case _:
        print("Opción desconocida")
```

_ es el caso por defecto. | permite alternativas. No hace falta break entre casos.

BUCLES: FOR Y WHILE

```
for marca in ["Seat", "Ford"]:
    print(marca)

n = 3
while n > 0:
    print(n)
    n -= 1
```

for recorre elementos. **while** repite mientras la condición sea verdadera.
break: sale del bucle más cercano. **continue**: pasa a su siguiente iteración. **pass**: no hace nada.

RANGE: SECUENCIA DE ENTEROS

range(inicio, fin, paso): no incluye fin.

```
list(range(4))         # [0, 1, 2, 3]
list(range(2, 5))      # [2, 3, 4]
list(range(0, 7, 2))   # [0, 2, 4, 6]
list(range(3, 0, -1))  # [3, 2, 1]
for i in range(3):
    print(i)
```

Inicio por defecto: 0; paso: 1. Paso 0 no es válido. range no es una lista; list(...) la materializa.

ENUMERATE, ZIP Y DICCIONARIOS

```
marcas = ["Seat", "Ford"]
precios = [12000, 15000]
for i, marca in enumerate(marcas, start=1):
    print(i, marca)
for marca, precio in zip(marcas, precios):
    print(marca, precio)
for clave, valor in {"km": 12000}.items():
    print(clave, valor)
```

enumerate: posición y valor. **zip**: agrupa por posición y, por defecto, termina con la secuencia más corta.

COMPRENSIONES

```
dobles = [x * 2 for x in range(4)]
pares = [x for x in range(6) if x % 2 == 0]
tabla = {x: x * 2 for x in range(3)}
```

FUNCIONES: DEFINIR, LLAMAR, DEVOLVER

```
def total(precio: float, unidades=1):
    return precio * unidades
importe = total(20, 3)      # por posición
importe = total(20, unidades=3)
def extremos(datos):
    return min(datos), max(datos)
menor, mayor = extremos([3, 8, 5])
```

Parámetro: nombre; **argumento**: valor enviado. **return** devuelve; **print** muestra. Sin return: None. Variables internas: locales. : float orienta, no valida.

ARGS, KWARGS, LAMBDA, MAP Y FILTER

***args**: argumentos en una tupla. ****kwargs**: argumentos con nombre en un diccionario.

```
def sumar(*args): return sum(args)
def ficha(**kwargs): return kwargs
doble = lambda x: x * 2
list(map(doble, [1, 2, 3]))  # [2, 4, 6]
list(filter(lambda x: x > 0, [-1, 2]))
```

lambda: función de una expresión. map transforma; filter filtra según una condición.

ITERABLE E ITERADOR

```
it = iter([10, 20])
next(it)             # 10
next(it)             # 20
next(it, None)       # None: agotado
```

Iterable: se puede recorrer. Iterador: se consume al avanzar. next(it) agotado lanza StopIteration. zip, map y filter son iteradores.

No entra en el examen del 28 de octubre

CLASES, OBJETOS Y HERENCIA

```
class Coche:
    def __init__(self, marca):
        self.marca = marca
    def tipo(self):
        return "Coche"
class Electrico(Coche):
    def tipo(self):
        return super().tipo() + " eléctrico"
c = Electrico("Seat")
print(c.marca, c.tipo())
```

self: objeto; **marca**: parámetro; **self.marca**: atributo. Heredar reutiliza; redefinir sobrescribe; super() permite ampliar. Si defines otro __init__, llama al del padre con super().__init__(...). **Polimorfismo**: mismo método, distintos tipos.

CLASE ABSTRACTA

```
from abc import ABC, abstractmethod
class Motor(ABC):
    @abstractmethod
    def arrancar(self):
        pass
```

No se puede instanciar mientras queden métodos abstractos pendientes de implementar.

TRY / EXCEPT: GESTIONAR ERRORES

```
try:
    km = int(input("Kilómetros: "))
    print(km)
except ValueError:
    print("Escribe un entero")
```

Captura el error esperado. Opcionales: **else** (si no hay error) y **finally** (se ejecuta siempre).